МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина
“Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №1

Выполнил:
Студент группы ИУ5-35Б
Борисенко М.В.
Преподаватели:
Гапанюк Ю.Е.
Нардид А.Н

Москва 2026

Задание

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия: 1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко многим. Пример классов данных для предметной области Сотрудник-Отдел: 1. Класс «Сотрудник», содержащий поля:

- ID записи о сотруднике;
- Фамилия сотрудника;
- Зарплата (количественный признак);
- ID записи об отделе. (для реализации связи один-ко-многим) 2. Класс «Отдел», содержащий поля:
- ID записи об отделе;
- Наименование отдела. 3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:
- ID записи о сотруднике;
- ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Листинг кода программы

main.py

```
from operator import itemgetter


class Book:
    def __init__(self, id, title, price, shop_id):
        self.id = id
        self.title = title
        self.price = price
        self.shop_id = shop_id


class Shop:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class BookShop:
    def __init__(self, shop_id, book_id):
        self.shop_id = shop_id
        self.book_id = book_id


shops = [
    Shop(1, 'Академкнига'),
    Shop(2, 'Альпина-Букс'),
    Shop(3, 'Библио-Глобус'),
    Shop(11, 'Абзац'),
    Shop(22, 'Буквоед'),
]

books = [
    Book(1, 'Чистый код', 1200, 1),
    Book(2, 'Изучаем C++', 2500, 2),
    Book(3, 'Совершенный код', 1800, 3),
    Book(4, 'Паттерны проектирования', 1500, 3),
    Book(5, 'Алгоритмы и структуры данных', 950, 3),
]

books_shops = [
    BookShop(1, 1),
    BookShop(2, 2),
    BookShop(3, 3),
    BookShop(3, 4),
    BookShop(3, 5),
    BookShop(11, 1),
    BookShop(22, 2),
    BookShop(1, 3),
```

```
    BookShop(11, 4),
    BookShop(22, 5),
]


def main():
    print('Вариант запросов Г, Вариант предметной области 16')

    one_to_many = [(b.title, b.price, s.name)
                   for s in shops
                   for b in books
                   if b.shop_id == s.id]

    many_to_many_temp = [(s.name, bs.shop_id, bs.book_id)
                         for s in shops
                         for bs in books_shops
                         if s.id == bs.shop_id]

    many_to_many = [(b.title, b.price, shop_name)
                    for shop_name, shop_id, book_id in many_to_many_temp
                    for b in books if b.id == book_id]

    print('\nЗадание Г1')
    res_11 = {}
    for s in shops:
        if s.name.startswith('A'):
            s_books = [title for title, _, shop_name in one_to_many if
shop_name == s.name]
            res_11[s.name] = s_books
    print(res_11)

    print('\nЗадание Г2')
    res_12_unsorted = []
    for s in shops:
        s_books = list(filter(lambda i: i[2] == s.name, one_to_many))
        if len(s_books) > 0:
            s_prices = [price for _, price, _ in s_books]
            res_12_unsorted.append((s.name, max(s_prices)))

    res_12 = sorted(res_12_unsorted, key=itemgetter(1), reverse=True)
    print(res_12)

    print('\nЗадание Г3')
    res_13 = sorted(many_to_many, key=itemgetter(2))
    print(res_13)

if __name__ == '__main__':
    main()
```

Пример работы проекта

```
Вариант запросов Г, Вариант предметной области 16

Задание Г1
{'Академкнига': ['Чистый код'], 'Альпина-Букс': ['Изучаем C++'], 'Абзац': []}

Задание Г2
[('Альпина-Букс', 2500), ('Библио-Глобус', 1800), ('Академкнига', 1200)]

Задание Г3
[('Чистый код', 1200, 'Абзац'), ('Паттерны проектирования', 1500, 'Абзац'), ('Чистый код', 1200,
 'Академкнига'), ('Совершенный код', 1800, 'Академкнига'), ('Изучаем C++', 2500, 'Альпина-Букс'),
  ('Совершенный код', 1800, 'Библио-Глобус'), ('Паттерны проектирования', 1500, 'Библио-Глобус'),
  ('Алгоритмы и структуры данных', 950, 'Библио-Глобус'), ('Изучаем C++', 2500, 'Буквоед'),
 ('Алгоритмы и структуры данных', 950, 'Буквоед')]

Process finished with exit code 0
```